4.1 对话技巧与命令

> **生成模型**：GPT-5.4 (openai/gpt-5.4)
> **Token 消耗**：输入 ~45k tokens，输出 ~4k tokens（本节）

很多人第一次用 OpenClaw，会先把它当成“会写字的聊天机器人”。聊两句之后才发现，不对，它更像一个带操作能力的助理。你不是只在问它“知不知道”，而是在指挥它“去做一下”。这一章就从最直接的入口讲起：怎么和 OpenClaw 聊，怎么用聊天命令把它调到合适的工作状态，怎么让一次对话既顺手又高效。

这一节不讲源码细节，重点站在使用者视角。你可以把它理解成一份“对话驾驶手册”：什么时候该开新会话，什么时候该压缩上下文，什么时候该切模型，怎么下指令更容易得到靠谱结果。

4.1.1 先弄明白：什么叫“工具调用”

在普通聊天产品里，AI 能做的事基本只有一件：生成文字。你问它一个问题，它回你一段文字；你让它写代码，它回你一段代码；仅此而已。

OpenClaw 不一样。它可以在回答之前，先去调别的能力，比如读文件、执行 Bash、打开浏览器、推送 Canvas 界面。这个过程就叫**工具调用**（tool calling，也常叫 function calling）。

如果你不是技术背景，可以把它想成这样：

- 模型像大脑，负责理解你的话，决定下一步要干什么。
- 工具像手和脚，负责真的去执行动作。
- OpenClaw 像一个管家，检查这个动作能不能做、该不该做、结果再怎么交回给模型。

所以“工具调用”不是 AI 偷偷乱跑命令，而是它先明确提出：“为了完成这个任务，我想调用某个工具，参数是这些。” 然后系统再决定要不要放行。用户看到的结果，往往是“它先做了，再回来告诉你做成了什么”。

举个最简单的例子：

用户消息：

```
请帮我看看这个目录里有哪些 Markdown 文件，并按名字分类。
```

AI 内部可能做的事是：

1. 判断：光靠猜不行，得先读目录。
2. 调用读文件或 Bash 相关工具。
3. 拿到目录结果。
4. 再整理成自然语言回复。

你看到的是一段很顺的回答，但背后其实已经发生了“先观察，再回答”。这就是 OpenClaw 跟普通聊天模型拉开差距的地方。

4.1.2 会话不是废话，它决定了 AI 记得什么

OpenClaw 的对话不是孤立的。它会把同一个会话里的上下文串起来：你前面说过什么，后面往往会继续记着。这有好处，也有坑。

好处很明显：

- 你不用每次都重新介绍背景。
- 你可以连续推进一个任务，比如“先看代码，再改文档，再跑测试”。
- 它能沿着你前面的要求保持风格，比如“以后都用中文”“解释给本科生听”。

坑也很明显：

- 聊太久以后，上下文会变脏。
- 前面一个任务的残留要求，可能污染后面的任务。
- 你已经换题了，它还在沿用上一题的假设。

所以，高手用 OpenClaw，不是一直死撑在一个长会话里，而是会主动管理会话。

一个很实用的经验是：

- 同一个目标，尽量放在同一个会话里，方便连续推进。
- 换了目标，就果断开新会话。
- 感觉回答开始跑偏、重复、反应慢，就考虑压缩或重置。

这里你就会用到下面这些聊天命令。

4.1.3 `/new`：开个新坑，别把旧包袱背过来

`/new` 的作用很直白：开始一个新会话。

什么时候该用它？最常见有三种场景。

第一种，你彻底换任务了。刚才还在让它帮你调 Node 项目，现在想让它总结一篇论文，继续在原会话里聊，前面的工程背景只会变成噪音。

第二种，你发现上下文已经很乱。比如你先后让它解释代码、改文档、查网页、生成表格，信息积得太多，模型就容易抓错重点。

第三种，你想保留旧记录，但又不想受旧记录影响。这时候 `/new` 比 `/reset` 更舒服，因为它是开一条新的线，不是把原来的线清空。

例子：

```
用户：/new 帮我从零开始规划一个学生用的记账机器人
AI：已开始新会话。我先按“目标用户、核心功能、技术栈、最小可行版本”四个部分来拆。
```

这类用法很适合开题。你相当于一边创建新上下文，一边给它一个新的任务起点。

4.1.4 `/reset`：还在这个话题里，但把上下文清一下

`/reset` 和 `/new` 很像，但不是一回事。

- `/new` 更像“新建一个聊天窗口”。
- `/reset` 更像“这个窗口还留着，但把上下文重置”。

如果你还想围绕同一主题继续聊，只是觉得上下文已经脏了，`/reset` 就很合适。

比如你一直在做一个课程项目，前面试过三种方案，聊得很乱，最后决定回到第一版思路。这时没必要开全新会话，因为主题还是那个项目；但你需要把之前的噪音扔掉。

例子：

```
用户：/reset 现在不要再考虑 Electron 方案了，我们只做 Web 版
AI：已重置当前会话。我接下来只按 Web 版来分析，不再带入 Electron 的前提。
```

这句话的关键，不是“重置”两个字，而是你顺手把新的边界条件也告诉它了。这样比只发一个 `/reset` 更好，因为你等于立即写入了新的起跑线。

4.1.5 `/compact` ：不是清空，而是“压缩重点”

`/compact` 很值得学。它适合那种“我不想丢掉上下文，但也不想继续带着一大包聊天记录跑”的情况。

你可以把它理解成：让 OpenClaw 先把当前对话压成一份更短的摘要，再继续往下聊。

这在长任务里特别有用，比如：

- 一整个下午都在做同一个课程作业
- 连续几轮都在改同一篇文档
- 先让它调研，再让它写方案，再让它落到执行步骤

这时候用 `/compact` ，通常能保住主线信息，又把很多已经没价值的细枝末节扔掉。

更好的用法，是给压缩一个方向：

```
用户：/compact 只保留我们已经确认的架构决策、待办项和风险
AI：已压缩上下文，后续我会优先基于这三类信息继续。
```

这其实很像你在带一个人开会：“前面聊散了，现在大家收一下，只记结论、待办和风险。” AI 也吃这一套。

4.1.6 `/think` ：控制它愿意花多少脑力

`/think` 可以理解成“思考档位”。简单说，就是让模型多想一点，还是快一点。

对日常用户来说，不用死记所有档位，记住一个原则就够了：

- 简单问题，低思考，出得快。
- 复杂任务，高思考，少走弯路。

什么算复杂？通常包括：

- 多步骤规划
- 代码排错
- 方案比较
- 需要权衡利弊的选择题
- 要结合多个工具一起做的任务

例子：

```
用户：/think high
用户：帮我设计一个图书馆座位预约系统，先给出最小可行版本，再说明后续怎么扩展
AI：我会按 MVP、数据结构、接口、前端页面、扩展路线来拆。
```

而如果你只是想问：

```
用户：/think low
用户：帮我解释一下什么是 REST API，用本科生能懂的话讲
```

这就很合理。别拿大锤砸钉子。思考越重，不代表越好，只代表更慢、更贵，也可能更啰嗦。

4.1.7 `/verbose`：让它多说还是少说

`/verbose` 决定回复是紧凑还是展开。

很多人一开始的误区是：只要 AI 更详细，就一定更有帮助。其实不是。要看你现在处于哪个阶段。

- 你在扫信息、快速试错，适合简洁。
- 你在学习原理、建立理解，适合详细。
- 你在执行具体步骤，适合“短结论 + 必要细节”。

例子：

```
用户：/verbose on
用户：解释一下 OpenClaw 里会话和工具为什么会互相影响
AI：好的，我会把“上下文”“工具结果残留”“任务边界”三件事分开讲。
```

另一个场景是反过来：

```
用户：/verbose off
用户：直接告诉我接下来三步做什么，不要展开
AI：1. 先开新会话。2. 明确任务边界。3. 再让它调用工具执行。
```

你会发现，很多时候不是 AI 不够聪明，而是你没有把输出密度调到合适档位。

4.1.8 `/model`：不同模型，脾气真不一样

`/model` 用来查看或切换当前模型。

别把模型切换想得太玄学。它本质上是给同一个 OpenClaw 换一个“脑子”。有的模型擅长推理，有的模型写作更顺，有的模型工具调用更积极，有的模型便宜适合日常杂活。

一个很现实的策略是：

- 日常问答、轻量整理，用便宜或快速模型。
- 写长文、做分析、复杂排错，用更强模型。
- 某个模型连续两次没理解你的意思，就换一个试试。

例子：

```
用户：/model
AI：当前模型：claude-sonnet-4

用户：/model gpt-5
AI：已切换到 gpt-5。
```

从使用体验上说，模型切换最常见的两个用途是：一，提质量；二，省成本。别迷信“永远只用一个最强模型”，那样常常没必要。

4.1.9 `/status`：别猜，让系统自己说现在是什么状态

`/status` 是很容易被低估的命令。它像一个仪表盘，让你知道当前会话到底在什么状态。

通常你能从它看到这些关键信息：

- 当前模型是谁
- 会话是不是刚重置过
- 某些运行选项是不是开着
- 上下文、用量或者相关状态摘要

为什么它重要？因为很多“AI 今天怎么怪怪的”问题，最后都不是 AI 抽风，而是状态不对。

比如：

- 你以为自己还在高思考，其实已经切回低档。
- 你以为切了模型，其实没切成。
- 你以为是新会话，结果还在旧上下文里。

例子：

```
用户：/status
AI：当前模型：gpt-5；思考级别：high；详细模式：off；当前会话已压缩一次。
```

养成一个小习惯：一旦感觉回复不对劲，先 `/status` 看一眼。省很多时间。

4.1.10 怎么提问，AI 才更容易把事做对

如果说聊天命令是在调“系统状态”，那你的自然语言就是在写“任务说明书”。说明书写得越清楚，OpenClaw 越容易干得准。

最有用的提问方式，不是客客气气地问一句“能不能帮我”，而是把下面四样东西说清楚：

1. **目标**：你最后想拿到什么。
2. **边界**：哪些不要做。
3. **输入**：它可以用哪些材料。
4. **输出格式**：你希望它怎么回。

比如下面这句就很一般：

```
帮我看看这个项目。
```

而下面这句就好很多：

```
帮我阅读这个课程项目目录，先判断它是前后端分离还是单体应用，再用 5 条告诉我主要模块。不要改代码，只做观察
```

差别在哪？后者把目标、范围、输出格式全交代了。AI 不用猜你到底想要代码审查、架构总结，还是部署建议。

4.1.11 一段实用对话：从乱聊到有效协作

下面这段模拟对话，很接近真实使用场景。

```
用户：我想做一个宿舍值日排班小工具。
AI：可以，我先帮你拆需求。

用户：/verbose on
用户：目标用户是 6 人宿舍，最好手机上也能看。
AI：那我先从使用场景、最小功能、技术方案三块来讲。

用户：/think high
用户：先别写代码，帮我比较“纯前端网页”和“微信小程序”两种方案。
AI：我会重点比较开发成本、分享方式、维护难度和登录需求。

用户：/compact 只保留已经确认的需求和两个方案的优缺点
AI：已压缩，后续我会基于这些结论继续。

用户：/new 现在我们单独开一条会话，只讨论纯前端网页版
AI：已开始新会话。你现在的项目前提是：6 人宿舍值日排班，优先手机访问，方案限定为纯前端网页。
```

这段对话里，没有一条命令是“炫技”。每一条都在做很具体的事：调节输出密度、提高思考档位、压缩上下文、切开主题。这就是命令真正的价值。

4.1.12 用户视角的会话管理经验

如果你只记几条经验，我建议记下面这些。

第一，**一个会话只服务一个目标**。别把“查资料”“写文档”“改代码”“做排班表”全塞进同一条线里。

第二，**命令和自然语言要配合着用**。比如 `/reset` 后面最好顺手补一句新的边界条件；`/compact` 后面最好说明保留什么。

第三，**先定输出，再让它开工**。你要的是清单、表格、分步计划，还是最终文案？提前说，省来回。

第四，**感觉跑偏就及时切会话**。不要硬救一个已经污染得很重的上下文。

第五，**把 AI 当协作者，不是算命先生**。它越清楚你要什么，越容易给出可执行结果。

本节小结

1. 工具调用的核心意思是：模型负责判断该做什么，工具负责真的去执行，OpenClaw 负责管控这个过程。

2. 会话会持续积累上下文，所以 `/new` 、 `/reset` 、 `/compact` 本质上是在管理“AI 记住什么、忘掉什么”。
3. `/think` 、 `/verbose` 、 `/model` 、 `/status` 不是高级玩家专属，它们就是日常调节效率和质量的几个旋钮。
4. 高质量提问通常要把目标、边界、输入和输出格式说清楚，这比单纯“问得礼貌”更重要。
5. 真正好用的 OpenClaw，不是一直硬聊，而是会用命令把会话修剪成适合当前任务的样子。